Project – Airline Reservation System

Object Oriented Programming

(CSEG1044)

Phase 2 : UML And Package and Design

Members:-

Yash Jangir – 590014858

Parth Choudhary-590016913

Dushyant Singh-590012592

Soham Sharma-590017041

Anuj Pandey-590017258

1. Package Structure Design

To maintain a clear separation between UI, service, DAO, and model classes, the project is organized into the following practical package layout:

- **model:** Contains all the core entity classes (Plain Old Java Objects or POJOs). These classes represent the data structures of the application and have no knowledge of the database or the UI.
- **dao (Data Access Object):** Contains classes exclusively responsible for communicating with the database. All SQL queries (INSERT, SELECT, UPDATE, DELETE) reside strictly in this package.
- **service:** Contains the core business logic and validation rules. It acts as a bridge between the ui and dao layers.
- **ui:** Contains all the Java Swing components (Frames, Panels, Dialogs, Event Listeners) that the user interacts with.
- **util:** Contains global utility classes, such as the database connection manager.

2. Class Responsibilities

Based on the guidelines, here is the short explanation of class responsibilities for the classes that should be considered early:

Model Layer Classes:

- **Route:** Stores route details (ID, origin, destination, distance, estimated duration).
- **Passenger:** Stores passenger details (ID, name, email, passport number, nationality, date of birth).
- **Flight:** Stores flight schedule details (Flight ID, associated Route ID, airplane model, capacity, departure/arrival dates and times, ticket price).
- **Booking:** Links a Passenger to a Flight and tracks the booking status (Pending, Confirmed, Cancelled).
- **Seat Allocation:** Stores the schedule details (seat ID, seat number, class type, check-in status) for a specific confirmed booking.

Service Layer Classes:

- **FlightService:** Handles business logic related to flights, such as validating departure dates, checking capacity for new bookings, and interacting with the FlightDao to save or fetch records.
- *(Additional Services: BookingService, RouteService will handle their respective entity logic).*

DAO Layer Classes:

- **RouteDao:** Executes SQL operations specifically for the Route table in the database.
- **BookingDao:** Executes SQL operations for the Booking table, such as inserting new bookings or updating a status to "Confirmed".
- *(Additional DAOs: FlightDao, PassengerDao, SeatAllocationDao will handle their respective tables).*

3. UML Class Relationships

This section outlines the relationships we should draw in our UML Class Diagram.

- **Route -> Flight:** A Route can have multiple Flights (1-to-Many).
- **Passenger -> Booking:** A Passenger can submit multiple Bookings for different flights (1-to-Many).
- **Flight -> Booking:** A Flight can receive multiple Bookings from different passengers (1-to-Many).
- **Booking -> Seat Allocation:** A confirmed Booking has exactly one Seat Allocation (1-to-1).

Layer Dependency: The ui classes depend on service classes. The service classes depend on dao classes and model classes. The dao classes depend on model classes to return database results as objects.

4. Adherence to Design Warnings

Our architectural design strictly avoids common anti-patterns:

- **No giant manager class:** We are not creating one giant manager class. Responsibilities are clearly divided among specific services like FlightService.
- **Clean Swing Handlers:** We strictly do not place SQL code inside Swing event handlers. Swing events will only call methods in the service layer.
- **Layered Validation:** We do not keep validation logic only in the UI. While the UI may do basic checks (e.g., empty fields), complex validations (e.g., duplicate booking checks) live in the service layer.
- **Purposeful Classes:** We do not create classes that have no real responsibility. Every class mapped above is essential for the application to function.

5. Review Checklist Justification

Before submitting, our design satisfies the phase checklist requirements:

- **Clear Roles:** Does every class have a clear role? Yes, roles are strictly divided by layers (Model = Data, DAO = DB, Service = Logic, UI = Interface).
- **Sensible Relationships:** Are relationships sensible for the workflow? Yes, the relationships perfectly model the real-world flow of Routes hosting Flights, and Passengers submitting Bookings.
- **Layer Separation:** Are service and DAO responsibilities separated? Yes, Services handle business rules while DAOs strictly handle database persistence.
- **Use Case Support:** Does the UML support the use cases from Phase 1? Yes, the proposed classes and their methods will directly allow for creating flights, recording bookings, confirming tickets, and allocating seats as outlined in Phase 1.